AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Engineering

Lab Report

Experiment # 02

Experiment Title: Familiarization with an STM32 microcontroller board, studying an LED blink test, and implementing a 2-push button LED activation circuit using an STM32 microcontroller board.

Date of Perform:	9 November 2025	**Date of Submission:**	15 November 2025
Course Title:	MICROPROCESSOR AND EMBEDDED SYSTEMS LAB		
Course Code:	EEE 4103	**Section:**	O
Semester:	Fall 2025-26	**Degree Program:**	BSc in CSE
Course Teacher:	**Dr. Md. Rukonuzzaman**		

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgment is made.
3. No part of this Assignment/Case Study has been written for me/us by any other person except where such collaboration has been authorized. by the concerned teacher and is acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared, and archived to detect plagiarism.
6. I/we permit a copy of my/our marked work to be retained by the Faculty Member for review by any internal/external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea, or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offense that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic, and visual forms, including electronic data, and oral presentations. Plagiarism occurs when the origin of the source is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or copy my/our work.

** Student(s) must complete all details except the faculty use part.*

** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group # 06

Sl No	Name	ID	PROGRAM	SIGNATURE
1	Mashkur Ibtesham Wazed	23-54266-3	BSc in CSE	
2	Arfanul Hoque Nehal	23-54270-3	BSc in CSE	
3	Chinmoy Guha	22-48056-2	BSc in CSE	
4	Suvra Chakraborty	22-48067-2	BSc in CSE	
5	Nafiur Rahman Siam	23-50834-1	BSc in CSE	
6	Mohammad Ansar Uddin	22-47975-2	BSc in CSE	

Faculty use only

FACULTY COMMENTS	**Marks Obtained**	
	Total Marks	

Table of Contents

Experiment Title: Familiarization with a microcontroller, the study of blink test using and implementation of a traffic control system using a microcontroller and Proteus Simulation platform.

Objectives:

The objectives of this experiment are to

1. Study the STM32 Microcontroller Board.
2. Learn basic programming commands of the STM32 Microcontroller Board.
3. Apply the coding techniques of the STM32 Microcontroller Board.
4. Implement the LED blink test using the STM32 Board.
5. Develop a circuit that includes two pushbuttons and two LEDs to control two LEDs separately by ensuring proper connections for efficient functionality.
6. Develop a simulation circuit model using Proteus incorporating an LED blink test and controlling two LEDs with two pushbuttons separately.

Equipment List:

1) STM32 Cube IDE (1.0.1 or any recent version)
2) STM32 Microcontroller board
3) Two LED lights (Red and Green)
4) Two 100 ohm resistors and two 10k ohm resistors
5) Two push button switches
6) Jumper wires

Circuit Diagram:

The circuit diagram for LED blink test:

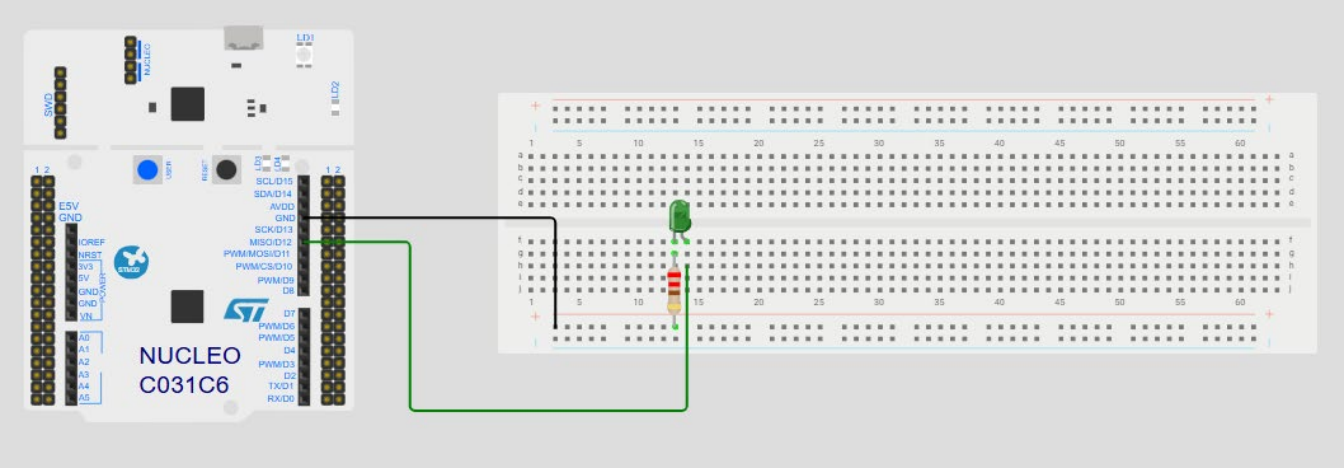

Fig. 1 Hardware circuit diagram for the blink test

The circuit diagram for Two push button LED control:

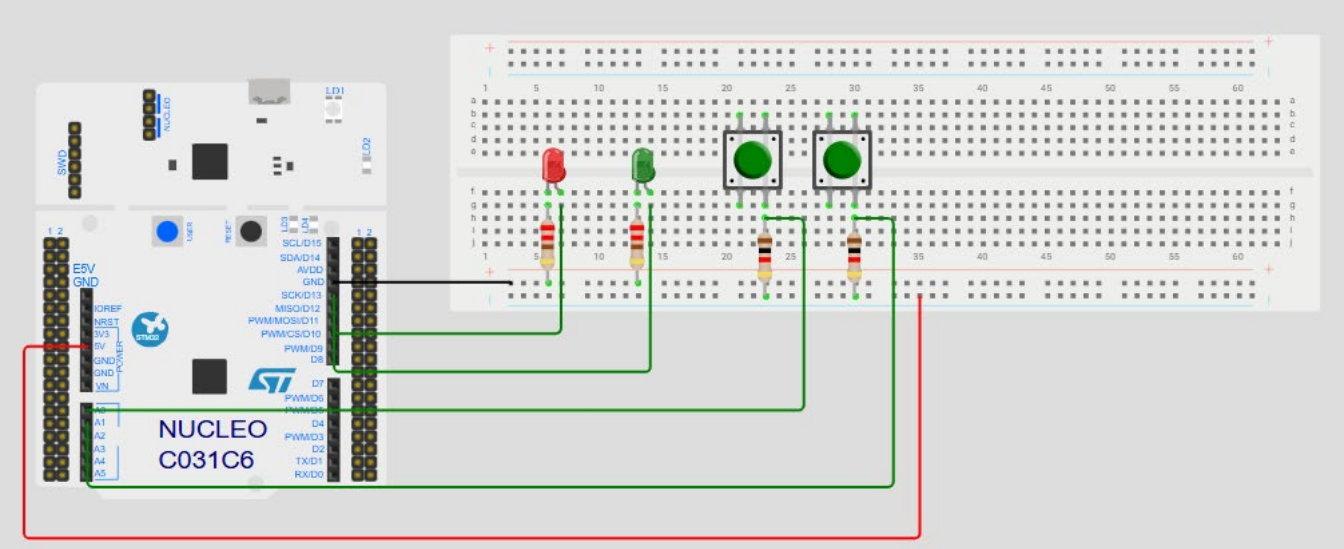

Fig. 2 Hardware circuit diagram for the Two push button LED control

Code/Program:
STM32CubeIDE .c code for blink test:

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
```

```
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
 UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */
```

```
  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */
          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
          HAL_Delay(1000);
    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
```

```
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE2);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 16;
  RCC_OscInitStruct.PLL.PLLN = 336;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
  RCC_OscInitStruct.PLL.PLLQ = 7;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief USART2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */
```

```
  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 115200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_6, GPIO_PIN_RESET);

  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
```

```
  /*Configure GPIO pins : PA0 PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : LD2_Pin PA6 */
  GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_6;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
```

```
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Explanation:

`HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);`

This line toggles the state of the GPIO pin PA5 (Pin 5 of GPIO port A). If the pin was previously set to LOW, it will be set to HIGH, and vice versa. In this case, it is used to turn an LED on and off.

`HAL_Delay(1000);`

This line introduces a delay of 1000 milliseconds (or 1 second), ensuring that the LED remains on or off for 1 second before toggling again.

STM32CubeIDE .c code for Two push button LED control:

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
```

```
/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
 UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();
```

```
  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0) {
                // Button 1 is pressed, turn on the Green LED and turn off the Red LED
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // Green LED ON
                }
        else {
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // Green LED OFF
                }
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == 0) {
                // Button 2 is pressed, turn on the Red LED and turn off the Green LED
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // Red LED ON
                }
        else {
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET); // Red LED OFF
                }
    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
```

```
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE2);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 16;
  RCC_OscInitStruct.PLL.PLLN = 336;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
  RCC_OscInitStruct.PLL.PLLQ = 7;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief USART2 Initialization Function
  * @param None
```

```
  * @retval None
  */
static void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 115200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
```

```
  HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_6, GPIO_PIN_RESET);

  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pins : PA0 PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : LD2_Pin PA6 */
  GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_6;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
```

```
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Explanation:

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == 0) {
        // Button 1 is pressed, turn on the Green LED and turn off the Red LED
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // Green LED ON
        }
else {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // Green LED OFF
        }
```

These lines of codes are responsible for checking if button 1, connected to PA0, is pressed or not. If pressed, the Green LED connected to PA5 lights up. If not pressed, the LED is off.

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == 0) {
        // Button 2 is pressed, turn on the Red LED and turn off the Green LED
                HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // Red LED ON
        }
else {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET); // Red LED OFF
        }
```

These lines of codes are responsible for checking if button 2, connected to PA1, is pressed or not. If pressed, the Red LED connected to PA6 lights up. If not pressed, the LED is off.

Hardware Output Results:

Hardware set up for LED blink test:

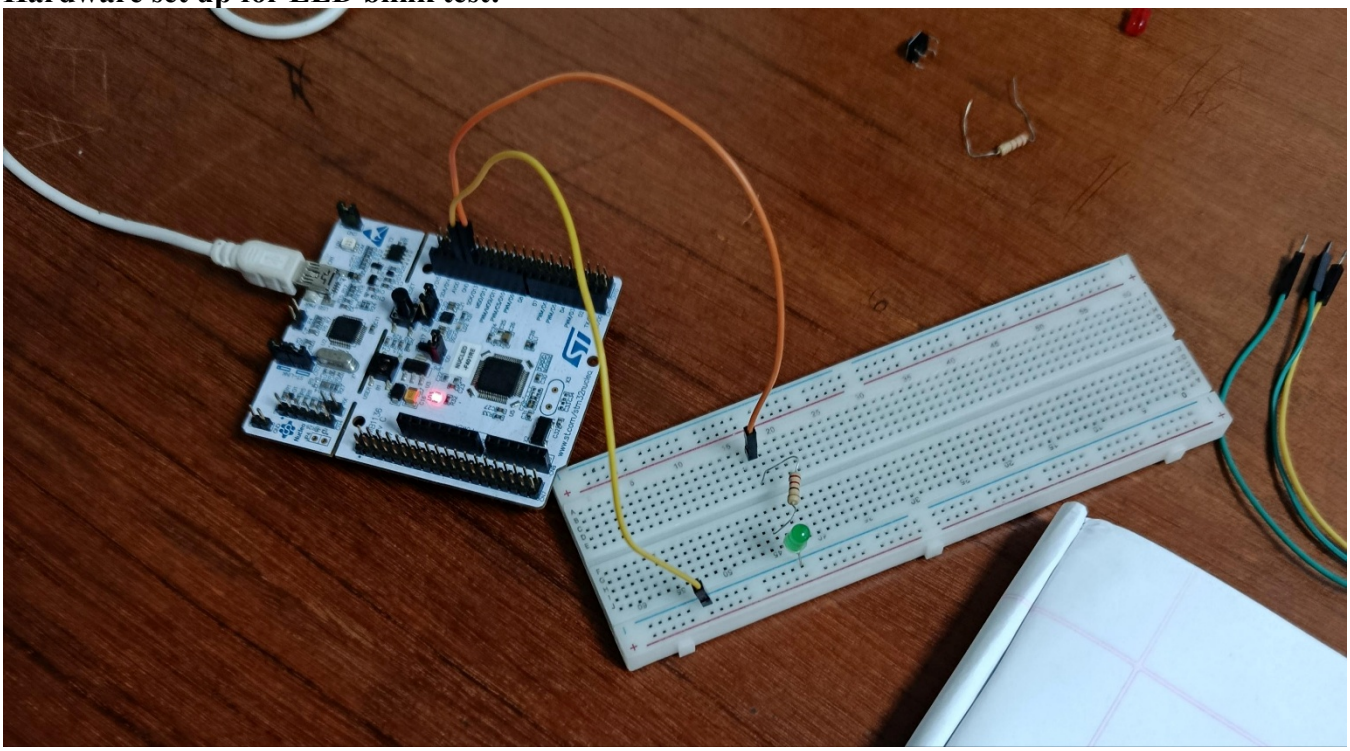

Fig. 3 Hardware set up for the LED blink test (LED Off)

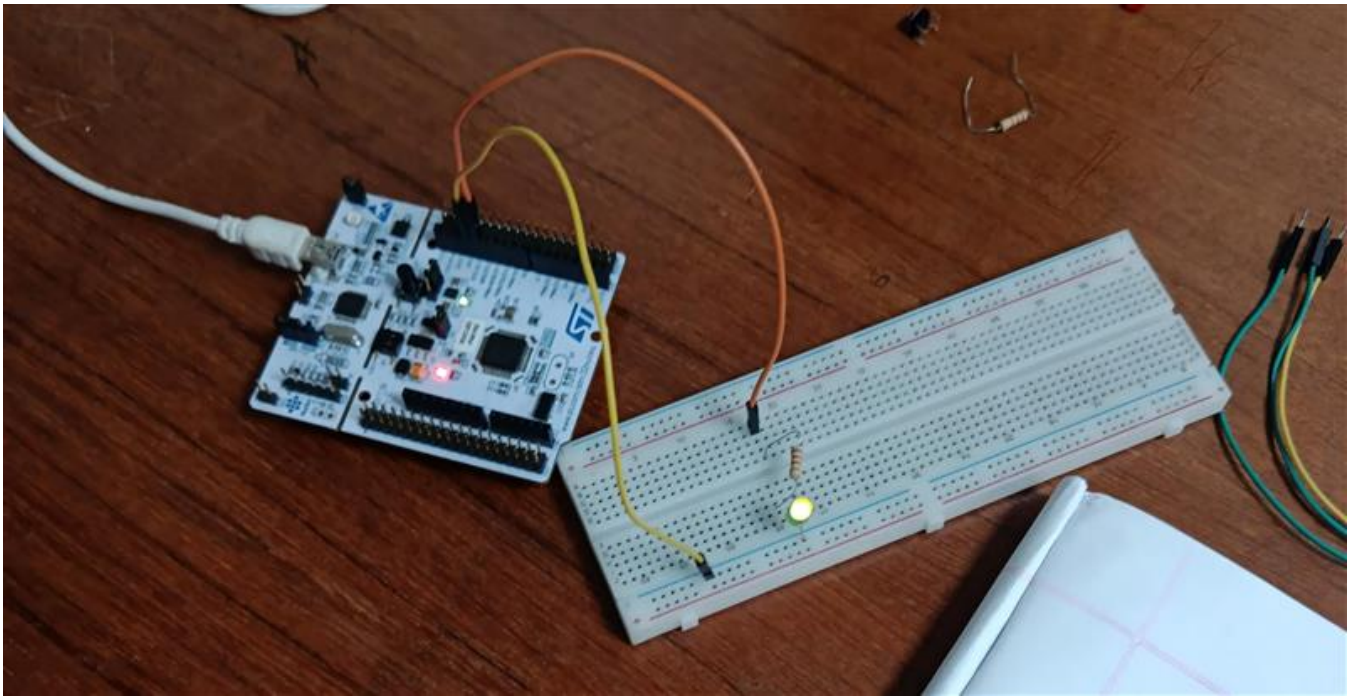

Fig. 4 Hardware set up for the LED blink test (LED On)

The STM32F401RE microcontroller board was connected to the laboratory computer, and the LED blink test program was uploaded and executed using STM32CubeIDE. The anode of the green LED on the breadboard was connected to the PA5 pin of the microcontroller through a jumper wire. The cathode of the LED was connected to a 220-ohm resistor, which was then connected to the GND pin of the microcontroller to complete the circuit.

Hardware set up for two push button LED control:

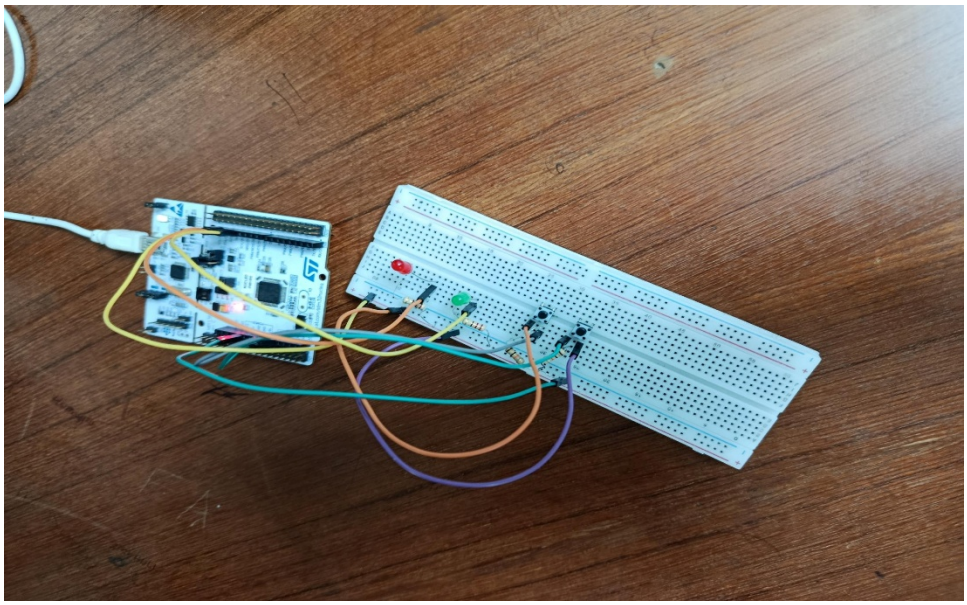

Fig. 5 Hardware set up for the Two push button LED control (Neutral state)

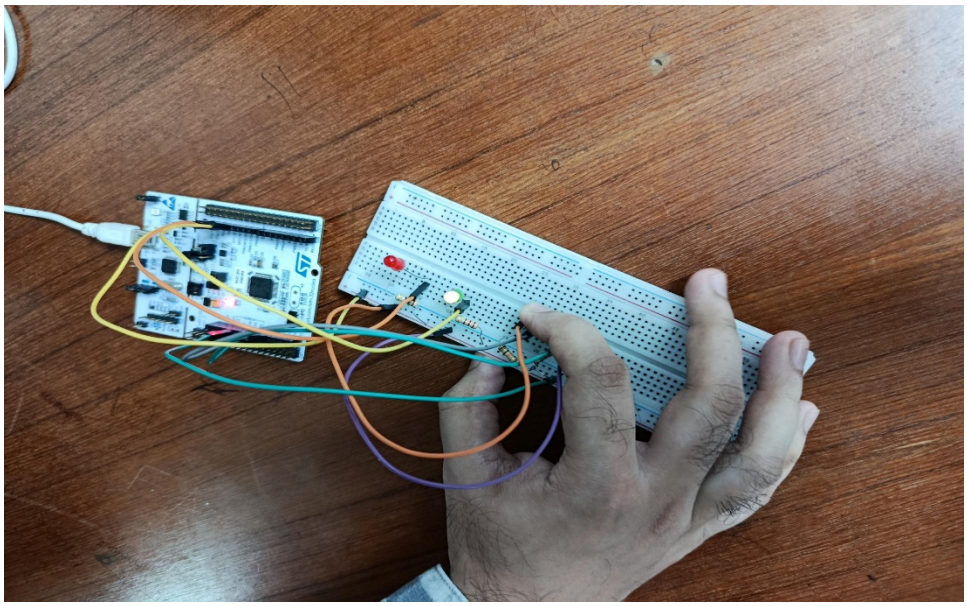

Fig. 6 Hardware set up for the Two push button LED control (Button 1 pressed, Green LED On, Red LED Off)

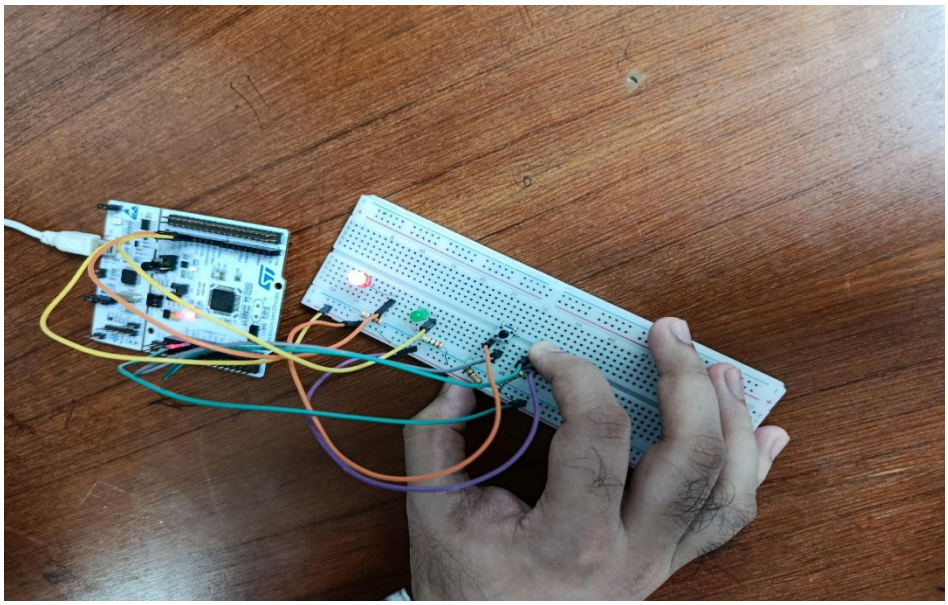

Fig. 7 Hardware set up for the Two push button LED control (Button 2 pressed, Red LED On, Green LED Off)

The STM32F401RE microcontroller was again connected to the laboratory computer, and the corresponding program for LED and push-button control was uploaded via STM32CubeIDE. Two LEDs, one red and one green, were placed on the breadboard. The anode of each LED was connected to separate GPIO output pins on the microcontroller using jumper wires, while their cathodes were each connected to ground through individual 220-ohm resistors.

Two push-button switches were also mounted on the breadboard. One terminal of each switch was connected to a dedicated GPIO input pin on the microcontroller, while the opposite terminal was connected to ground through separate 1k-ohm resistors to ensure a defined logic LOW when the button is not pressed. Additional jumper wires were used to complete the connections between the buttons and the microcontroller's ground and input pins. This configuration allowed the microcontroller to read button presses and control the LEDs accordingly.

Experimental Output Results:

The Experimental Output Results and the Hardware Output Results above, are the same as the experiment does not produce any relevant numerical or quantitative data that can be tabulated and plotted onto a graph.

Simulation Output Results:

Proteus simulation for LED blink test:

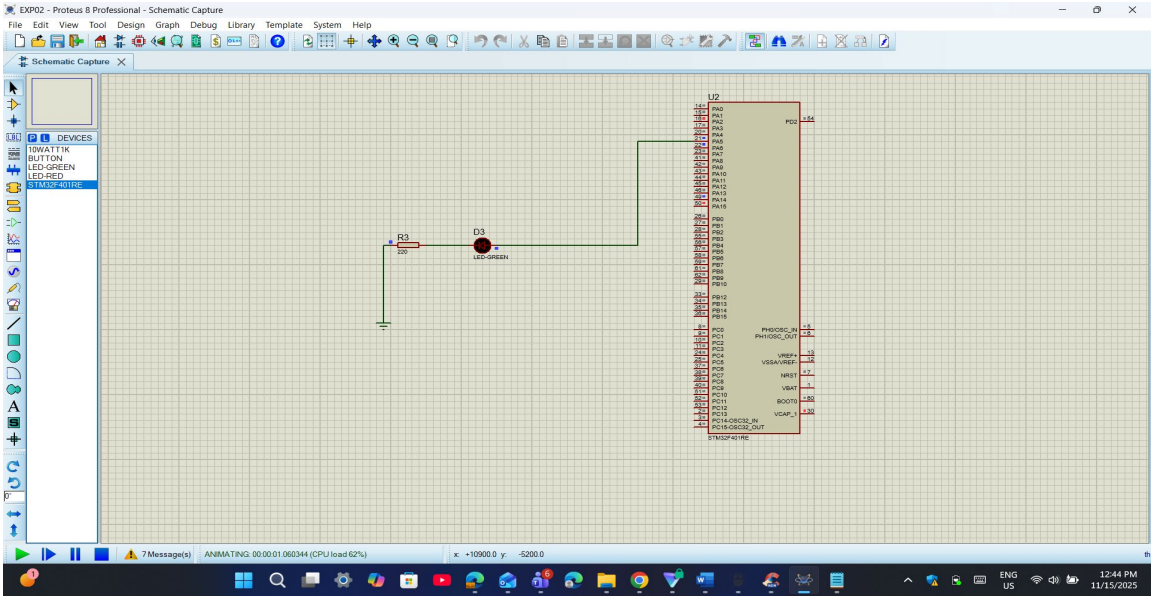

Fig. 8 Proteus simulation of the LED blink test (LED Off)

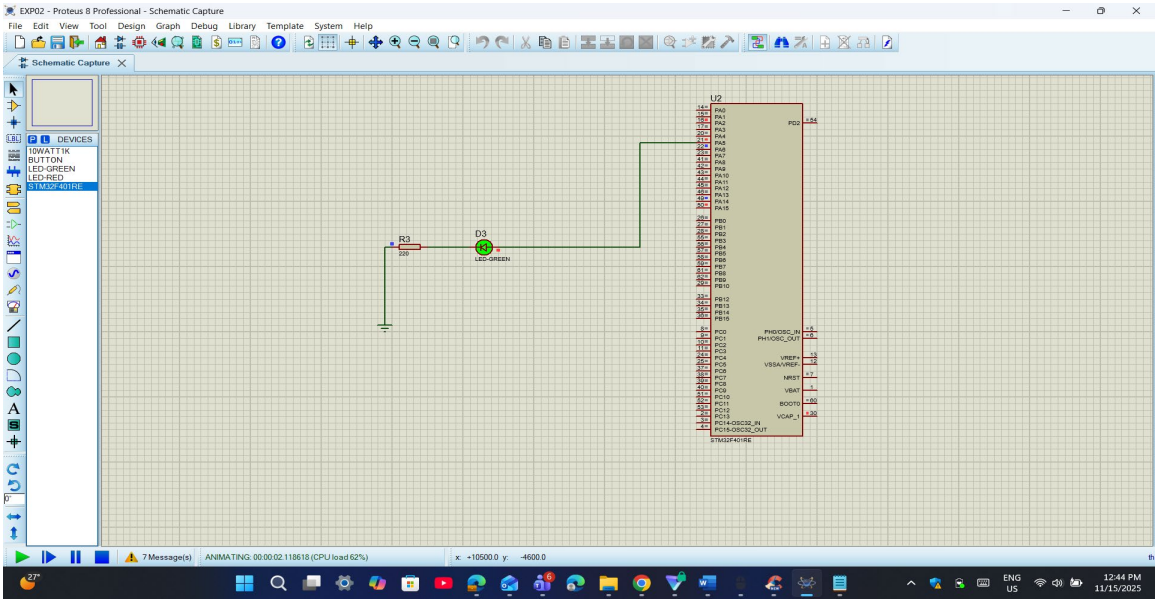

Fig. 9 Proteus simulation for the LED blink test (LED On)

Proteus simulation for LED control with push buttons:

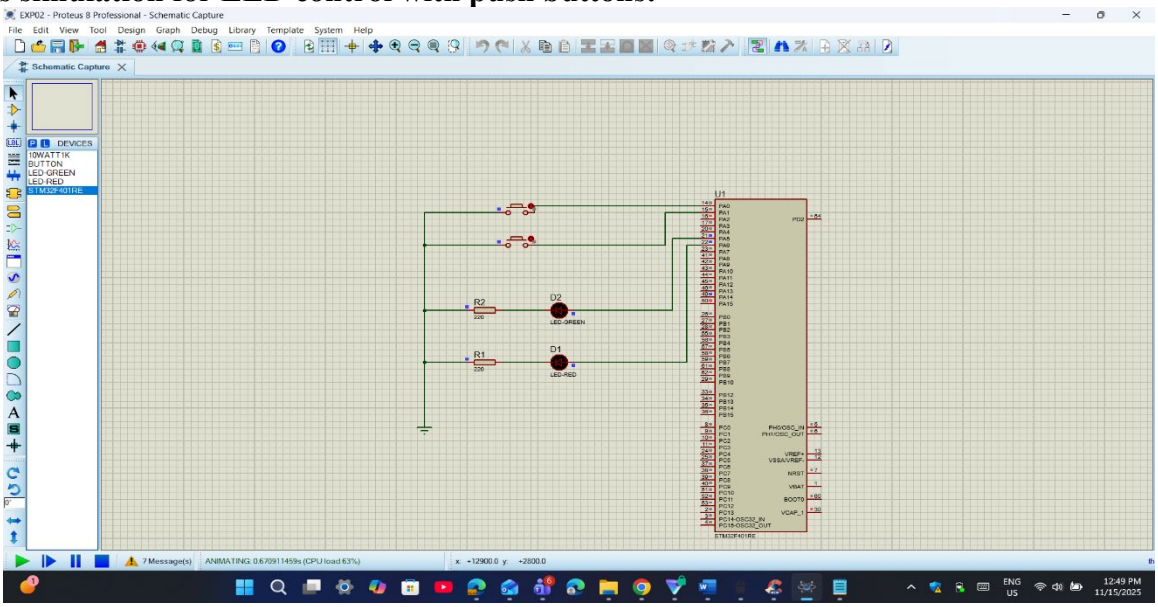

Fig. 10 Proteus simulation for the Two push button LED control (Neutral state)

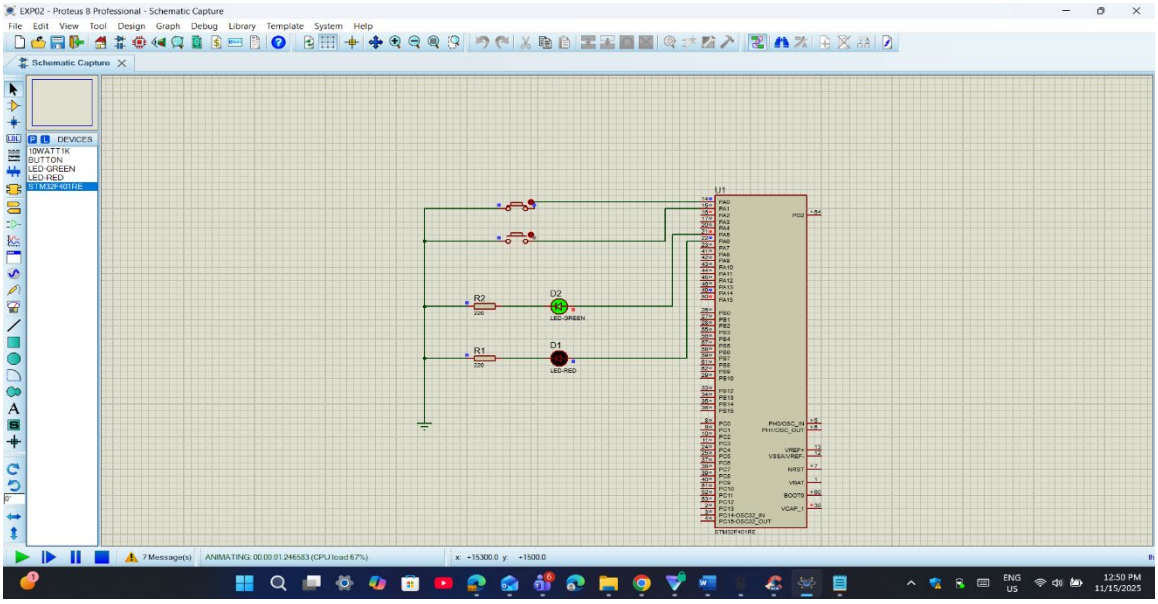

Fig. 11 Proteus simulation for the Two push button LED control (Button 1 pressed, Green LED On, Red LED Off)

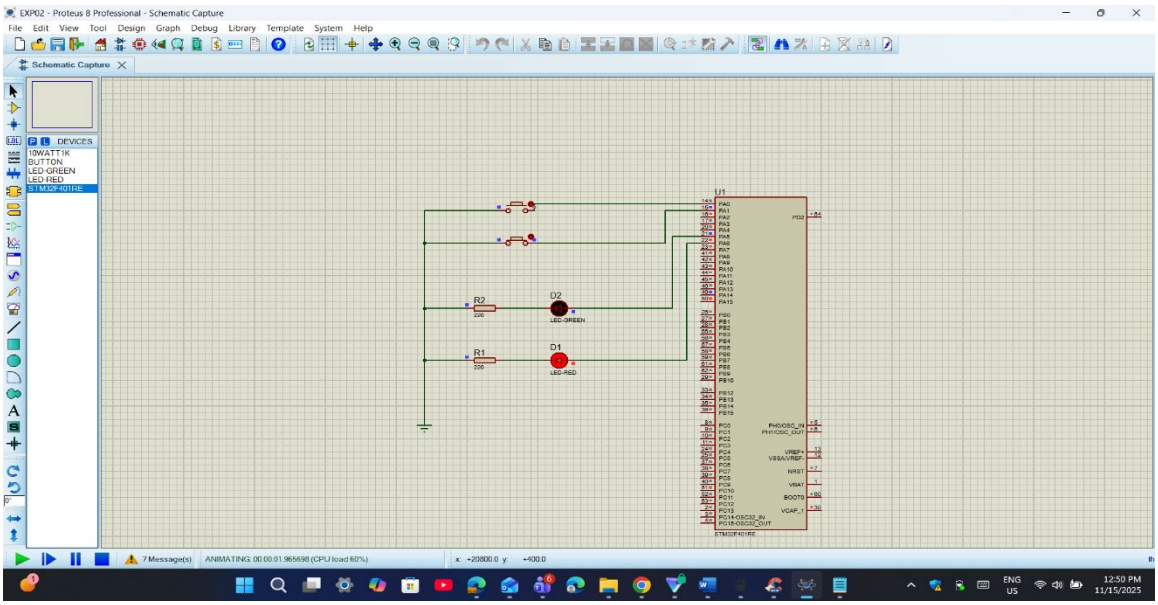

Fig. 12 Proteus simulation for the Two push button LED control (Button 1 pressed, Red LED On, Green LED Off)

Proteus 8 Professional simulation program was run, then a new project was created. The relevant circuit components were searched for and placed on the design grid, and connected accordingly to the experiment guidelines. Then the simulation was run, and screenshots of the relevant observations were taken for the LED blink test set up and the two push button LED control set up.

Answers to the Questions in the Lab Manual:

1. Include all codes and scripts in the lab report following the lab report writing template.

Codes for the LED blink test and the Two push button LED control have been provided above.

2. Draw the program flow chart of both codes and explain them.

Flowchart for blink test:

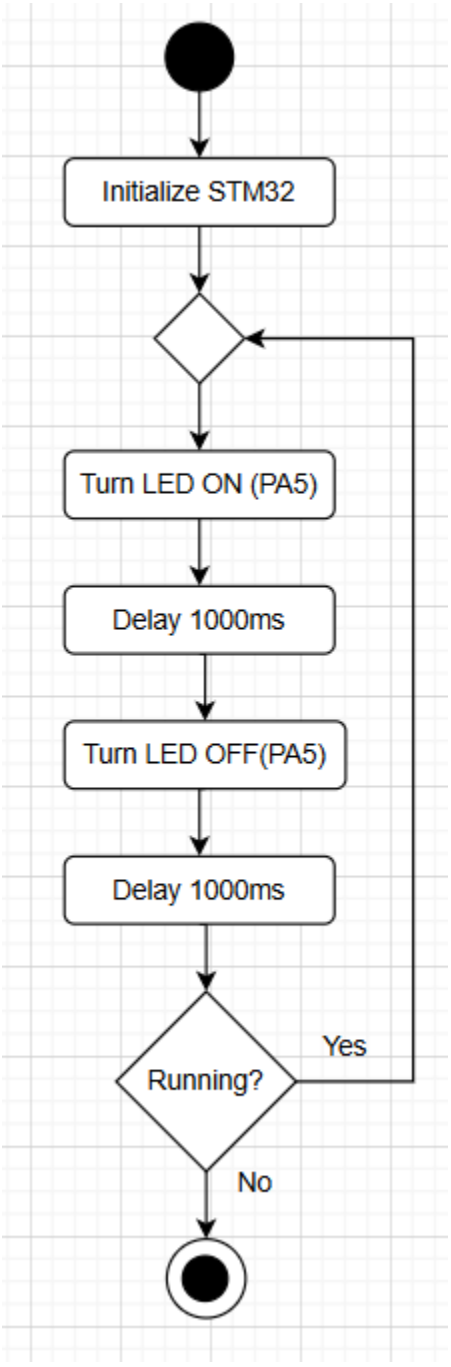

Fig. 13 Flowchart for LED blink test

i. The microcontroller turns the LED ON (PA5) by setting the output pin to a high logic level.

ii. It then pauses for a Delay of 1000ms (1 second), during which the LED remains on.

iii. After the delay, the microcontroller turns the LED OFF (PA5) by setting the output pin to a low logic level.

iv. Another Delay of 1000ms is applied, during which the LED remains off.

Flowchart for two push button LED control:

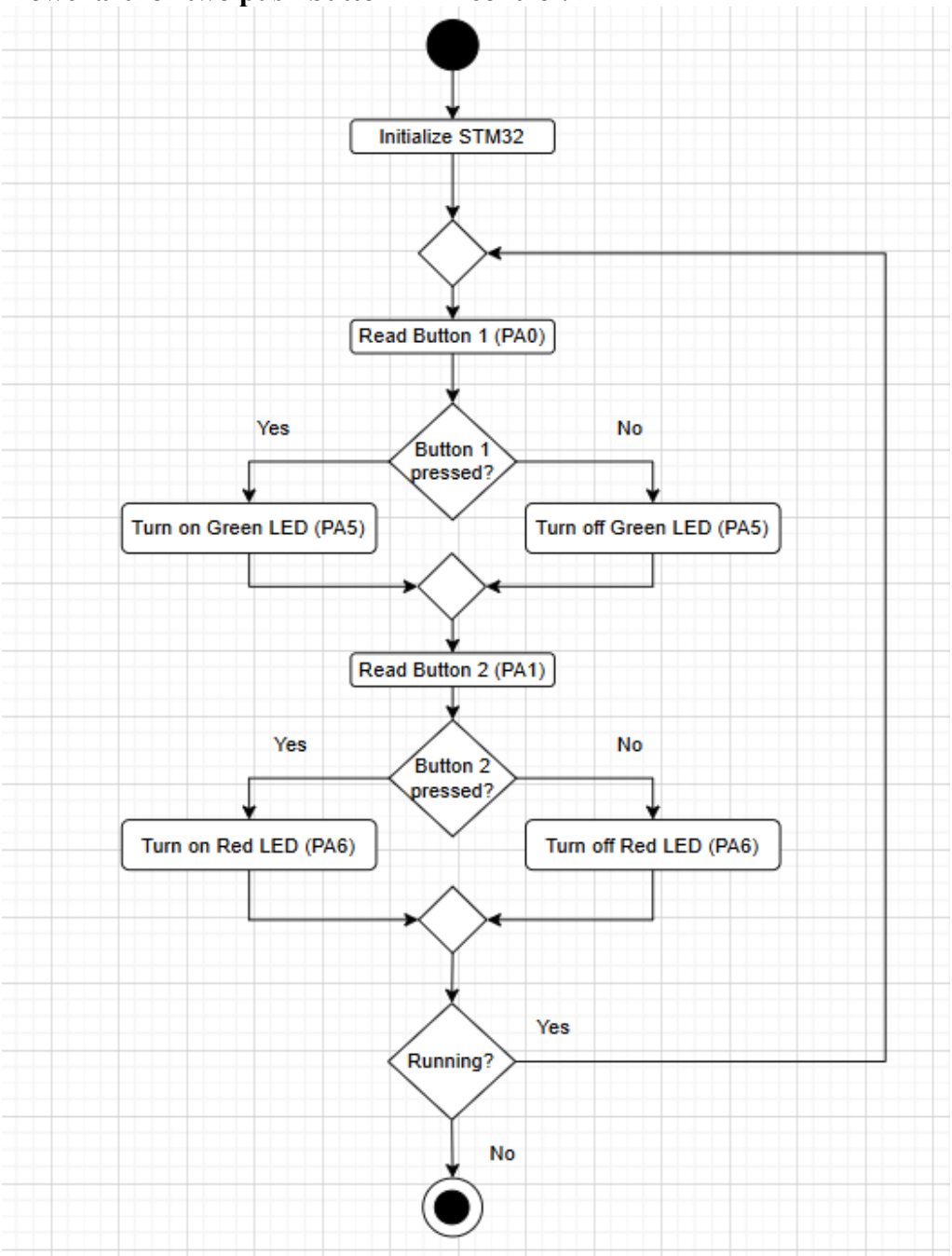

Fig. 14 Flowchart for two push button LED control

- For Button 1 (PA0):
    i. The microcontroller Reads Button 1 (PA0).
    ii. If the button is pressed (logic low, represented by "Yes"), it Turns on the Green LED (PA5).
    iii. If the button is not pressed (logic high, represented by "No"), it Turns off the Green LED (PA5).
- For Button 2 (PA1):
    i. The microcontroller then Reads Button 2 (PA1).
    ii. If the button is pressed (logic low, represented by "Yes"), it Turns on the Red LED (PA6).
    iii. If the button is not pressed (logic high, represented by "No"), it Turns off the Red LED (PA6).

3. Include the Proteus simulation of the blink program and two push button LED control.

The screenshots of the Proteus simulations for the blink program and the two push button LED control have been provided above.

Discussion:

By carrying out this experiment, we have learned how an STM32 microcontroller board is utilized, learning and applying the basic programming commands and coding techniques of the STM32 microcontroller board. And we learned how to generate corresponding .hex files for the codes on the STM32Cube IDE, which is then utilized in the Proteus software simulations for the circuit setups which were constructed via hardware in the laboratory. Carrying out the simulations aided in identifying potential errors in a risk-free environment which would be otherwise difficult to identify first-hand in the actual hardware setup.

References:

[1] https://www.st.com/en/evaluation-tools/nucleo-f401re.html for STM32F401RE datasheet.
[2] www.st.com.
[3] https://www.st.com/resource/en/user_manual/dm00105879-description-of-stm32f4-hal-and-ll drivers-stmicroelectronics.pdf.
[4] www.st.com/en/development-tools/stm32cubeide.html.